

GRAPHS

Unit 8

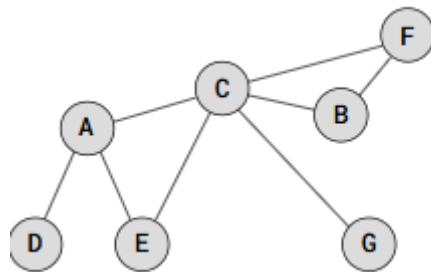
Presented By: Saroj Bhandari

Syllabus

- 8.4 Definition and Representation of Graphs, Graph Traversal, Minimum Spanning Trees: Kruskal and Prims Algorithm
- 8.5 Shortest Path Algorithms: Dijkstra Algorithm

Graph

- A Graph is a non-linear data structure consisting of vertices and edges.
- The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph.
- More formally a Graph is composed of a set of vertices(**V**) and a set of edges(**E**). The graph is denoted by **G(V, E)**.



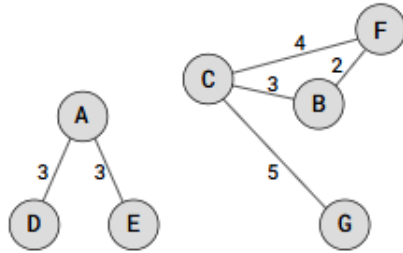
Applications of Graph:

- Social Networks: Each person is a vertex, and relationships (like friendships) are the edges. Algorithms can suggest potential friends.
- Maps and Navigation: Locations, like a town or bus stops, are stored as vertices, and roads are stored as edges. Algorithms can find the shortest route between two locations when stored as a Graph.
- Internet: Can be represented as a Graph, with web pages as vertices and hyperlinks as edges.
- Biology: Graphs can model systems like neural networks or the spread of diseases.

Graph Types

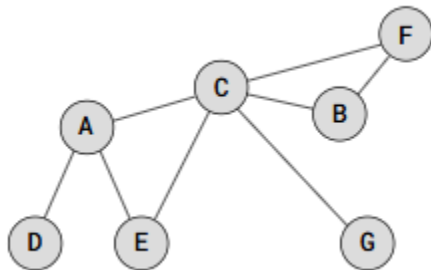
1) Weighted Graph

- ☒ Weighted
- ☐ Connected
- ☐ Directed
- ☐ Cyclic
- ☐ Loop



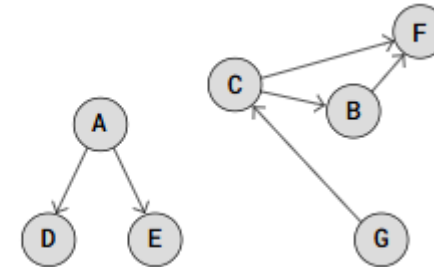
1) Connected Graph

- ☐ Weighted
- ☒ Connected
- ☐ Directed
- ☐ Cyclic
- ☐ Loop



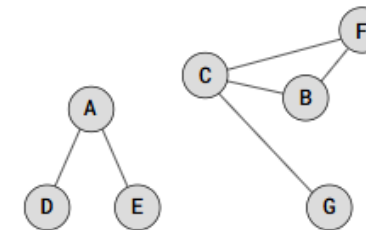
1) Directed Graph

- ☐ Weighted
- ☐ Connected
- ☒ Directed
- ☐ Cyclic
- ☐ Loop



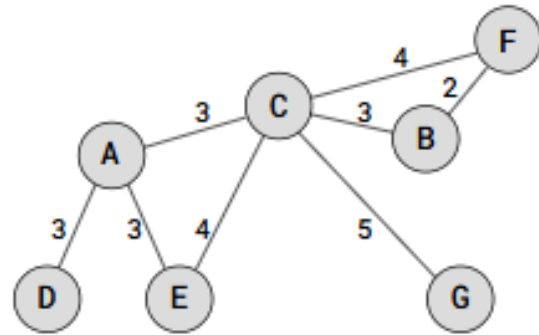
1) Cyclic Graph

- ☐ Weighted
- ☐ Connected
- ☐ Directed
- ☒ Cyclic
- ☐ Loop

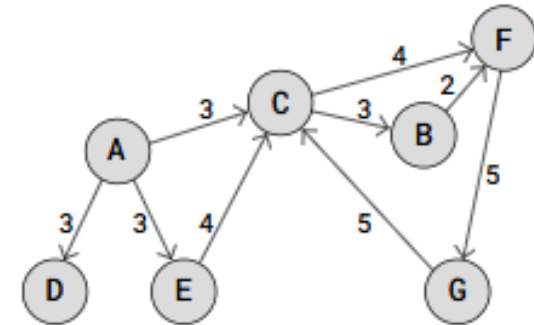


Lets **Combine** them and Take a look:

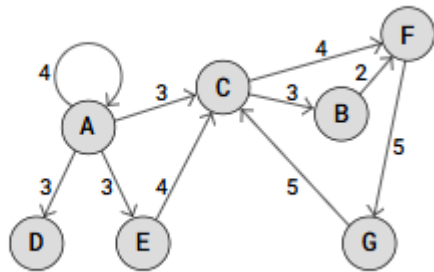
- ☒ Weighted
- ☒ Connected
- ☐ Directed
- ☒ Cyclic
- ☐ Loop



- ☒ Weighted
- ☒ Connected
- ☒ Directed
- ☒ Cyclic
- ☐ Loop



- ☒ Weighted
- ☒ Connected
- ☒ Directed
- ☒ Cyclic
- ☒ Loop



Understood? Lets define them

- A **weighted** Graph is a Graph where the edges have values. The weight value of an edge can represent things like distance, capacity, time, or probability.
- A **connected** Graph is when all the vertices are connected through edges somehow. A Graph that is not connected, is a Graph with isolated (disjoint) subgraphs, or single isolated vertices.
- A **directed** Graph, also known as a digraph, is when the edges between the vertex pairs have a direction. The direction of an edge can represent things like hierarchy or flow.
- A cyclic Graph is defined differently depending on whether it is directed or not:
- A **directed cyclic** Graph is when you can follow a path along the directed edges that goes in circles. Removing the directed edge from F to G in the animation above makes the directed Graph not cyclic anymore.
- An **undirected cyclic** Graph is when you can come back to the same vertex you started at without using the same edge more than once. The undirected Graph above is cyclic because we can start and end up in vertex C without using the same edge twice.
- A **loop**, also called a self-loop, is an edge that begins and ends on the same vertex. A loop is a cycle that only consists of one edge. By adding the loop on vertex A in the animation above, the Graph becomes cyclic.

Graph as an ADT

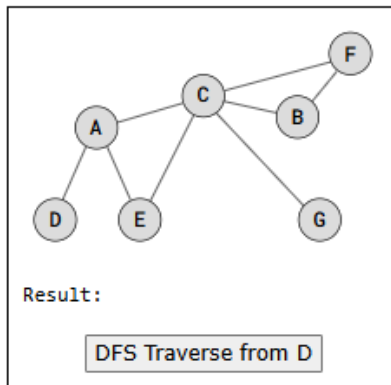
- A graph in computer science, as an Abstract Data Type (ADT), is a fundamental structure for representing relationships between entities.
- It consists of a set of nodes (vertices) and connections (edges) between them. Graphs can be directed or undirected, and edges can have weights representing costs or distances.
- This ADT is used to model and analyze networks in various applications.

Operations on a **Graph ADT**:

- ❑ `add_vertex(vert)`: Adds a new vertex to the graph.
- ❑ `add_edge(from_vert, to_vert, [weight])`: Adds an edge between two vertices, optionally with a weight.
- ❑ `get_vertex(vert_key)`: Retrieves a specific vertex from the graph.
- ❑ `get_vertices()`: Returns a list of all vertices in the graph.
- ❑ `has_edge(from_vert, to_vert)`: Checks if an edge exists between two vertices.
- ❑ `get_neighbors(vert)`: Returns a list of vertices adjacent to a given vertex.

Graph Traversal

- To **traverse a Graph** means to start in one vertex, and go along the edges to visit other vertices until all vertices, or as many as possible, have been visited.



- Understanding how a Graph can be traversed is important for understanding how algorithms that run on Graphs work.
- The two **most common ways a Graph can be traversed** are:
 - **Depth First Search (DFS)**
 - **Breadth First Search (BFS)**
- DFS is usually implemented using a Stack or by the use of recursion (which utilizes the call stack), while BFS is usually implemented using a Queue.

Depth First Search Traversal

- ❑ Depth first Search or Depth first traversal is a recursive algorithm for searching all the vertices of a graph or tree data structure. Traversal means visiting all the nodes of a graph.

- ❑ **Depth First Search Algorithm**

- ❑ A standard DFS implementation puts each vertex of the graph into one of two categories:

 - ❑ Visited

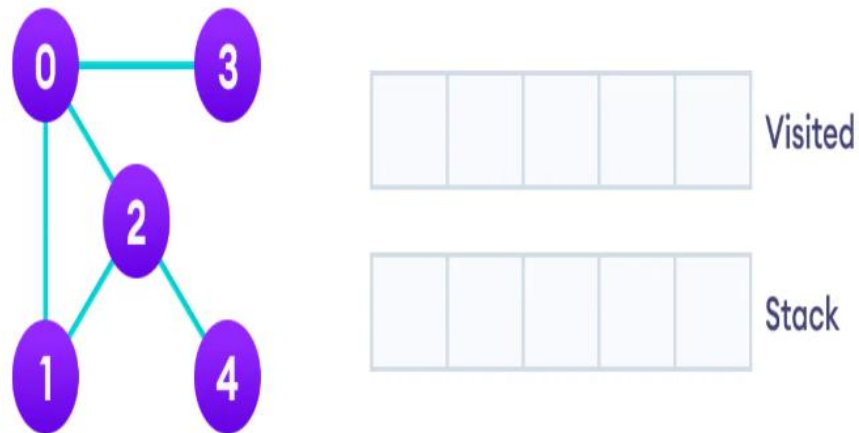
 - ❑ Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

Working Mechanism of DFS

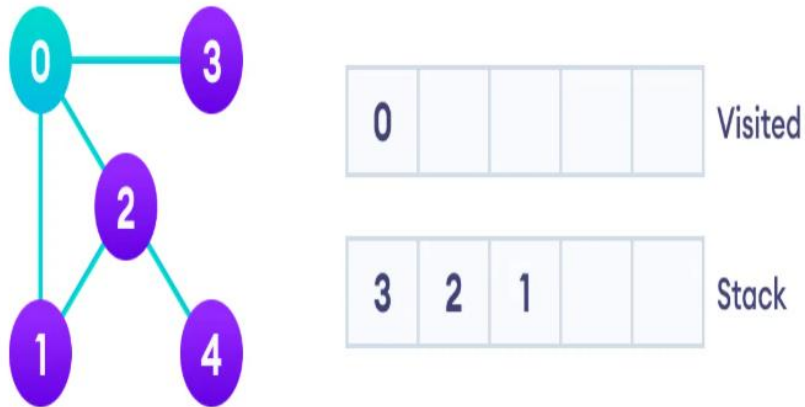
- ❑ The **DFS algorithm** works as follows:
 - ❑ Start by putting any one of the graph's vertices on top of a stack.
 - ❑ Take the top item of the stack and add it to the visited list.
 - ❑ Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
 - ❑ Keep repeating steps 2 and 3 until the stack is empty.

Depth First Search Example



We start from vertex 0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.

Depth First Search Example



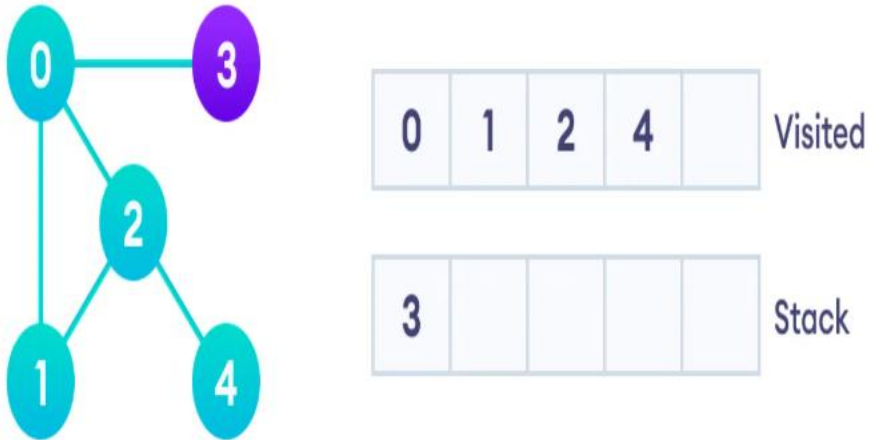
Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

Depth First Search Example



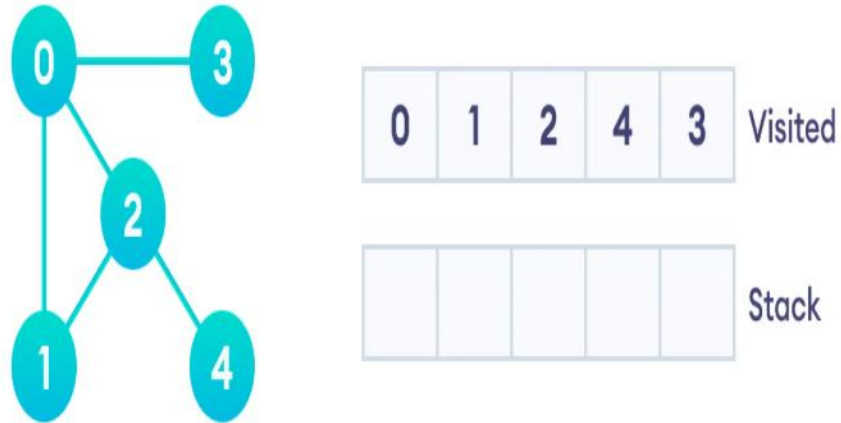
Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.

Depth First Search Example



After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

Depth First Search Example



After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

DFS Pseudocode (recursive implementation)

```
DFS(G, u)
    u.visited = true
    for each v ∈ G.Adj[u]
        if v.visited == false
            DFS(G,v)

init() {
    For each u ∈ G
        u.visited = false
    For each u ∈ G
        DFS(G, u)
}
```

DFS Implementation

- In Lab

Complexity of Depth First Search

- ❑ The time complexity of the DFS algorithm is represented in the form of $O(V + E)$, where V is the number of nodes and E is the number of edges.
- ❑ The space complexity of the algorithm is $O(V)$.

Application of DFS Algorithm

1. For finding the path
2. To test if the graph is bipartite
3. For finding the strongly connected components of a graph
4. For detecting cycles in a graph

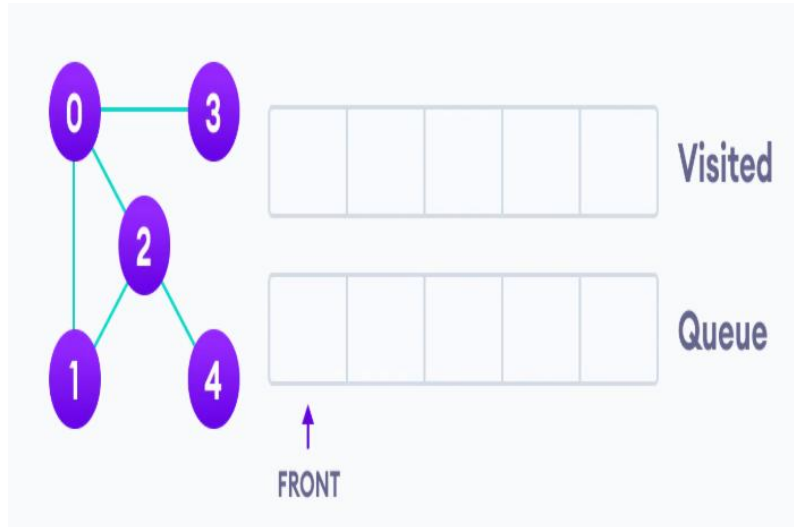
Breadth first search

- Traversal means visiting all the nodes of a graph. Breadth First Traversal or Breadth First Search is a recursive algorithm for searching all the vertices of a graph or tree data structure.

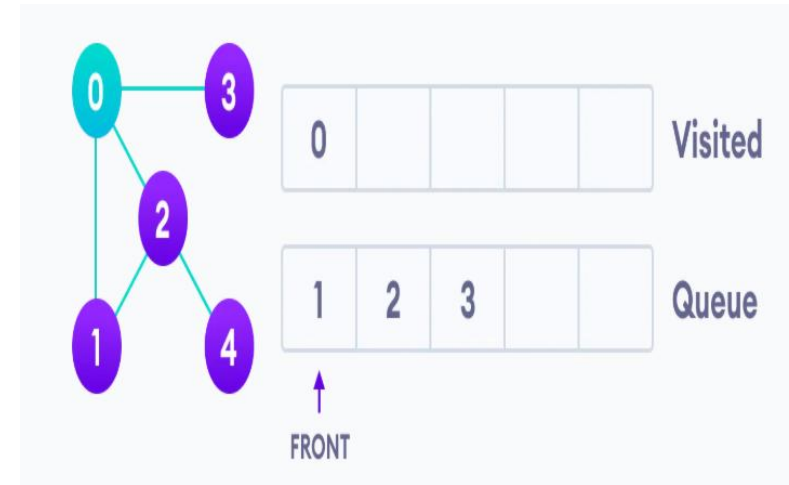
BFS algorithm

- A standard BFS implementation puts each vertex of the graph into one of two categories:
 - Visited
 - Not Visited
- The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.
- The algorithm works as follows:
 - Start by putting any one of the graph's vertices at the back of a queue.
 - Take the front item of the queue and add it to the visited list.
 - Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
 - Keep repeating steps 2 and 3 until the queue is empty.
 - The graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the BFS algorithm on every node

BFS example

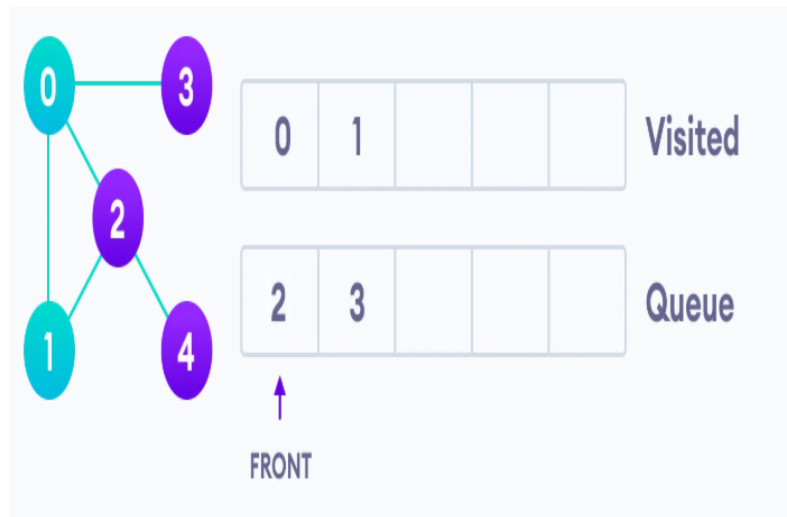


We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the queue.

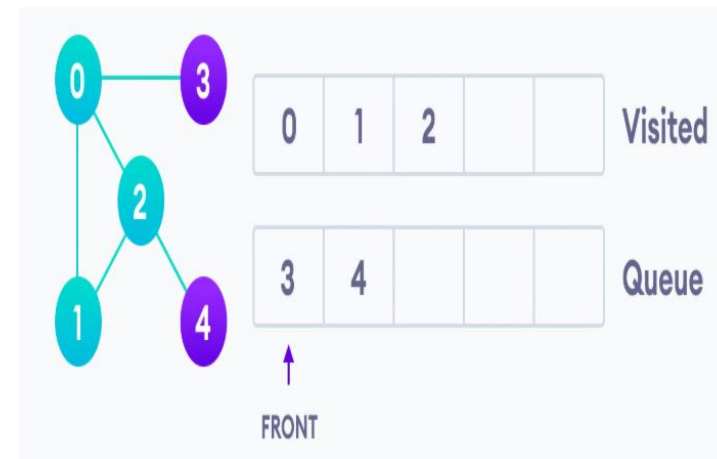


Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

BFS example

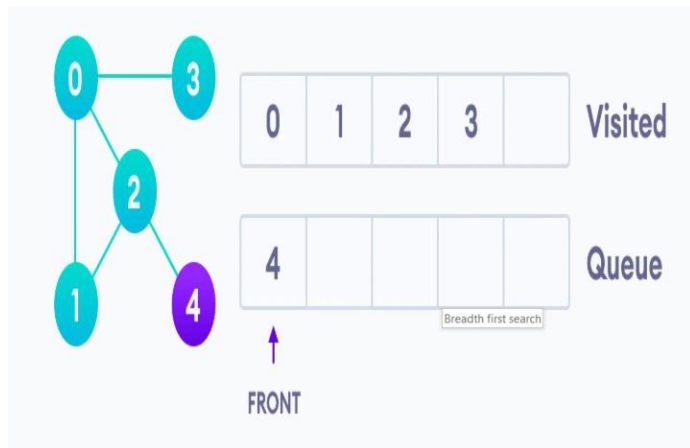


Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.

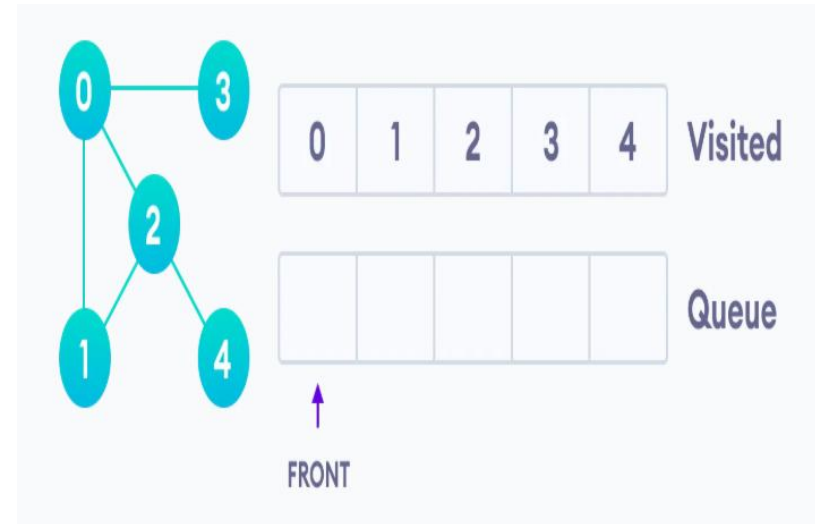


Visit 2 which was added to queue earlier to add its neighbours

BFS example



Only 4 remains in the queue since the only adjacent node of 3 i.e. 0 is already visited. We visit it.



Visit last remaining item in the queue to check if it has unvisited neighbors

BFS pseudocode

```
create a queue Q
mark v as visited and put v into Q
while Q is non-empty
    remove the head u of Q
    mark and enqueue all (unvisited) neighbours of u
```

BFS **Code** Implementation

- In Lab

BFS Algorithm Complexity

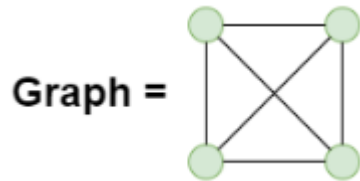
- ❑ The time complexity of the BFS algorithm is represented in the form of $O(V + E)$, where V is the number of nodes and E is the number of edges.
- ❑ The space complexity of the algorithm is $O(V)$.

BFS Algorithm Applications

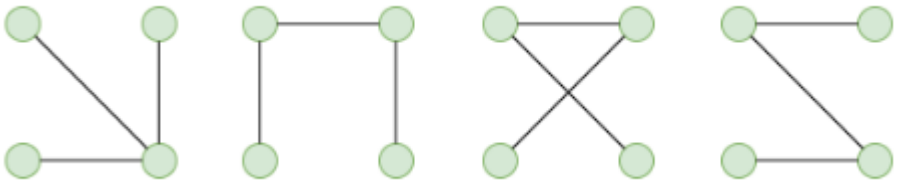
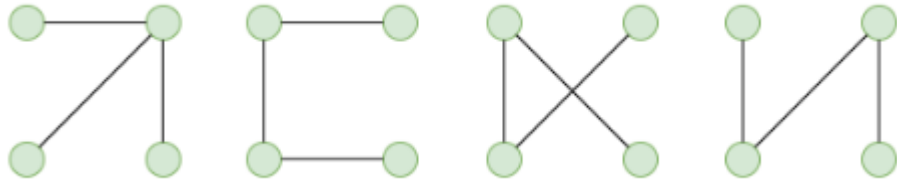
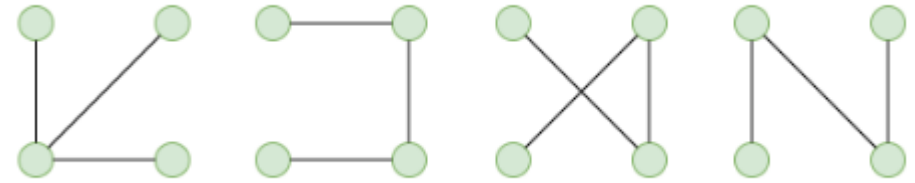
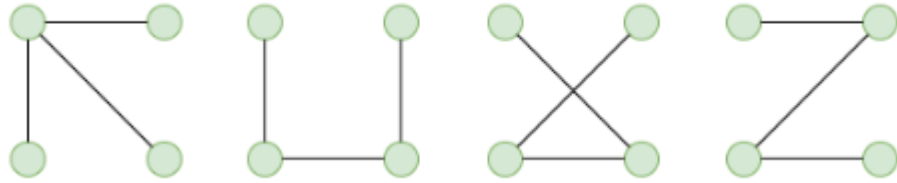
1. To build index by search index
2. For GPS navigation
3. Path finding algorithms
4. In Ford-Fulkerson algorithm to find maximum flow in a network
5. Cycle detection in an undirected graph
6. In [minimum spanning tree](#)

Spanning Tree

- *A spanning tree is a subset of Graph G , such that all the vertices are connected using minimum possible number of edges. Hence, a spanning tree does not have cycles and a graph may have more than one spanning tree.*
- **Example:** Lets construct a spanning tree form the given graph:



Spanning Tree



Properties of a **Spanning Tree**:

- A Spanning tree does not exist for a disconnected graph.
- For a connected graph having **N** vertices then the number of edges in the spanning tree for that graph will be **N-1**.
- A Spanning tree does not have any cycle.
- We can construct a spanning tree for a complete graph by removing **E-N+1** edges, where **E** is the number of Edges and **N** is the number of vertices.
- **Cayley's Formula:** It states that the number of spanning trees in a complete graph with N vertices is N^{N-2}
- For example: $N=4$, then maximum number of spanning tree possible $= 4^{4-2} = 4^2 = 16$ (shown in the above image).

Minimum Spanning Tree

- ❑ The weight of a spanning tree is determined by the sum of weight of all the edge involved in it.
- ❑ A minimum spanning tree (MST) is defined as a spanning tree that has the minimum weight among all the possible spanning trees.

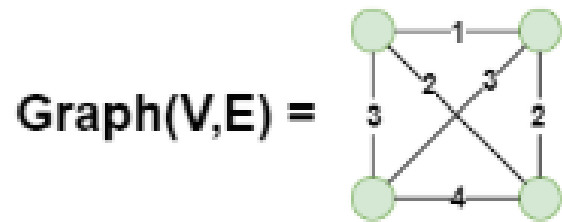
Minimum Spanning Tree for Directed Graph



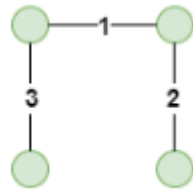
Properties of Minimum Spanning Tree:

- A minimum spanning tree connects all the vertices in the graph, ensuring that there is a path between any pair of nodes.
- An **MST** is **acyclic**, meaning it contains no cycles. This property ensures that it remains a tree and not a graph with loops.
- An **MST** with **V** vertices (where **V** is the number of vertices in the original graph) will have exactly **V - 1** edges, where **V** is the number of vertices.
- An **MST** is optimal for minimizing the total edge weight, but it may not necessarily be unique.
- The cut property states that if you take any cut (a partition of the vertices into two sets) in the original graph and consider the minimum-weight edge that crosses the cut, that edge is part of the **MST**.

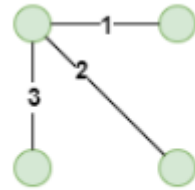
Minimum Spanning Tree of a Graph may not be Unique:



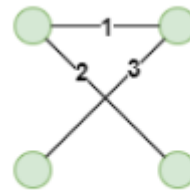
All Possible MST's of the above Graph



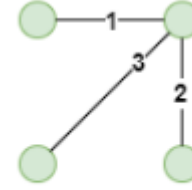
MST Cost =6



MST Cost =6



MST Cost =6



MST Cost =6

Algorithms to Find Minimum Spanning Tree of a Graph:

- There are several algorithms to find the minimum spanning tree from a given graph, some of them are listed below:
 - ❑ Kruskal's Minimum Spanning Tree (MST) Algorithm
 - ❑ Prim's Algorithm for Minimum Spanning Tree (MST)

Kruskal's Algorithm

- In **Kruskal's algorithm**, we sort all edges of the given graph in increasing order. Then it keeps on adding new edges and nodes in the MST if the newly added edge does not form a cycle.
- It picks the minimum weighted edge at first and the maximum weighted edge at last.
- Thus we can say that it makes a locally optimal choice in each step in order to find the optimal solution. Hence this is a Greedy Algorithm.
- Greedy is an algorithmic paradigm that builds up a solution **piece by piece**, always choosing the next piece that offers the most **obvious** and **immediate benefit**. Greedy algorithms are used for optimization problems.

How to find MST using Kruskal's algorithm?

Steps for finding MST using Kruskal's algorithm:

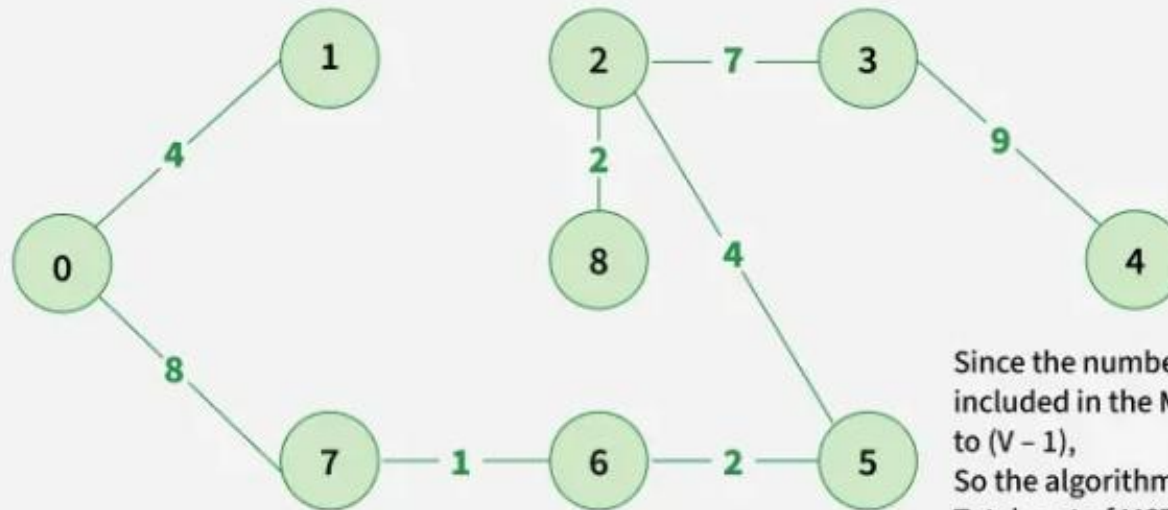
- ❑ Sort all the edges in a non-decreasing order of their weight.
- ❑ Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
- ❑ Repeat step 2 until there are $(V-1)$ edges in the spanning tree.

Example

The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having $(9 - 1) = 8$ edges.

10
Step

Final obtained graph minimum spanning tree.



Since the number of edges included in the MST equals to $(V - 1)$,
So the algorithm stops here.
Total cost of MST = (37)

Kruskal's Minimum Spanning Tree (MST) Algorithm

Time Complexity: $O(E * \log E)$ or $O(E * \log V)$

- Sorting of edges takes $O(E * \log E)$ time.
- After sorting, we iterate through all edges and apply the find-union algorithm. The find and union operations can take at most $O(\log V)$ time.
- So overall complexity is $O(E * \log E + E * \log V)$ time.
- The value of E can be at most $O(V^2)$, so $O(\log V)$ and $O(\log E)$ are the same. Therefore, the overall time complexity is $O(E * \log E)$ or $O(E * \log V)$
- **Auxiliary Space:** $O(E+V)$, where V is the number of vertices and E is the number of edges in the graph.

Prim's Algorithm for Minimum Spanning Tree (MST)

- Prim's algorithm is a Greedy algorithm like [Kruskal's algorithm](#). This algorithm always starts with a single node and moves through several adjacent nodes, in order to explore all of the connected edges along the way.
- The algorithm starts with an empty spanning tree.
- The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, and the other set contains the vertices not yet included.
- At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

Working of the Prim's Algorithm

Step 1: Determine an arbitrary vertex as the starting vertex of the MST. We pick o in the below diagram.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

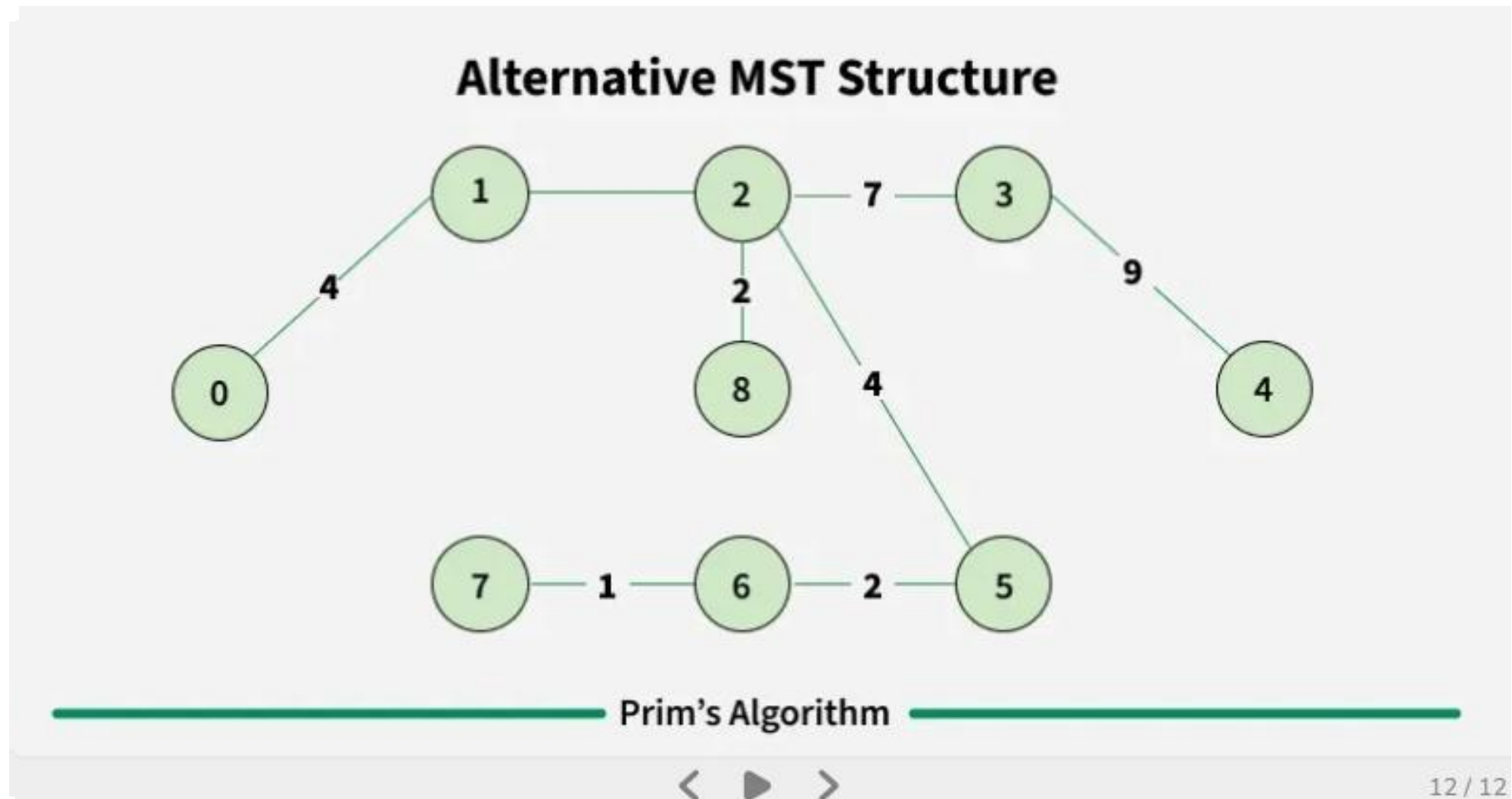
Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST. Since we consider only the edges that connect fringe vertices with the rest, we never get a cycle.

Step 6: Return the MST and exit

Illustration



Time and Space Complexity

- **Time Complexity:** $O(V^2)$, As, we are using adjacency matrix, if the input graph is represented using an adjacency list, then the time complexity of Prim's algorithm can be reduced to $O((E+V) * \log V)$ with the help of a binary heap.
- **Auxiliary Space:** $O(V)$

Advantage and Disadvantages

- **Advantages:**

- Prim's algorithm is guaranteed to find the MST in a connected, weighted graph.
- It has a time complexity of $O((E+V)*\log(V))$ using a binary heap or Fibonacci heap, where E is the number of edges and V is the number of vertices.
- It is a relatively simple algorithm to understand and implement compared to some other MST algorithms.

- **Disadvantages:**

- Like Kruskal's algorithm, Prim's algorithm can be slow on dense graphs with many edges, as it requires iterating over all edges at least once.
- Prim's algorithm relies on a priority queue, which can take up extra memory and slow down the algorithm on very large graphs.
- The choice of starting node can affect the MST output, which may not be desirable in some applications.

Dijkstra's Algorithm to find Shortest Paths from a Source to all

- ❑ Given a weighted undirected graph represented as an edge list and a source vertex src , find the shortest path distances from the source vertex to all other vertices in the graph.
- ❑ The graph contains V vertices, numbered from 0 to $V - 1$.

Note: The given graph does not contain any negative edge.

Algorithm

Step-by-Step Implementation

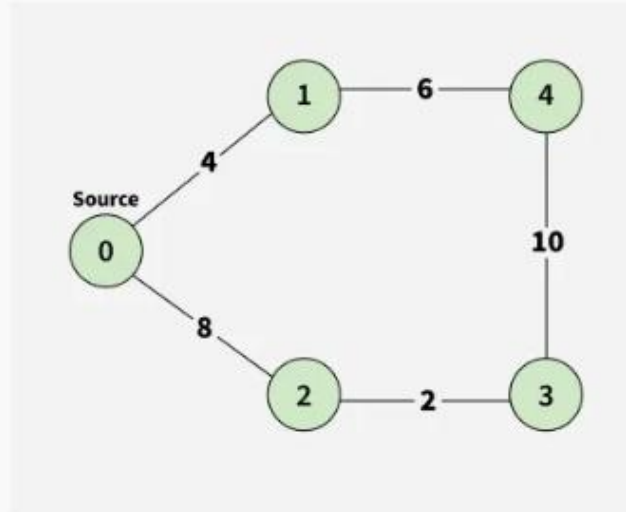
1. Set `dist[source]=0` and all other distances as `infinity`.
2. Push the source node into the min heap as a pair `<distance, node>` $\rightarrow$ i.e., `<0, source>`.
3. Pop the top element (node with the smallest distance).
 1. For each adjacent neighbor of the current node:
 2. Calculate the distance using the formula:
$$\text{dist}[v] = \text{dist}[u] + \text{weight}[u][v]$$

If this new distance is shorter than the current `dist[v]`, update it.

Push the updated pair `<dist[v], v>` into the min heap
4. **Repeat** step 3 until the min heap is empty.
5. Return the distance array, which holds the shortest distance from the source to all nodes.

Illustration

Input: $src = 0$, $V = 5$, $edges[][] = [[0, 1, 4], [0, 2, 8], [1, 4, 6], [2, 3, 2], [3, 4, 10]]$



Graph with 5 node

Output: 0 4 8 10 10

Explanation: Shortest Paths:

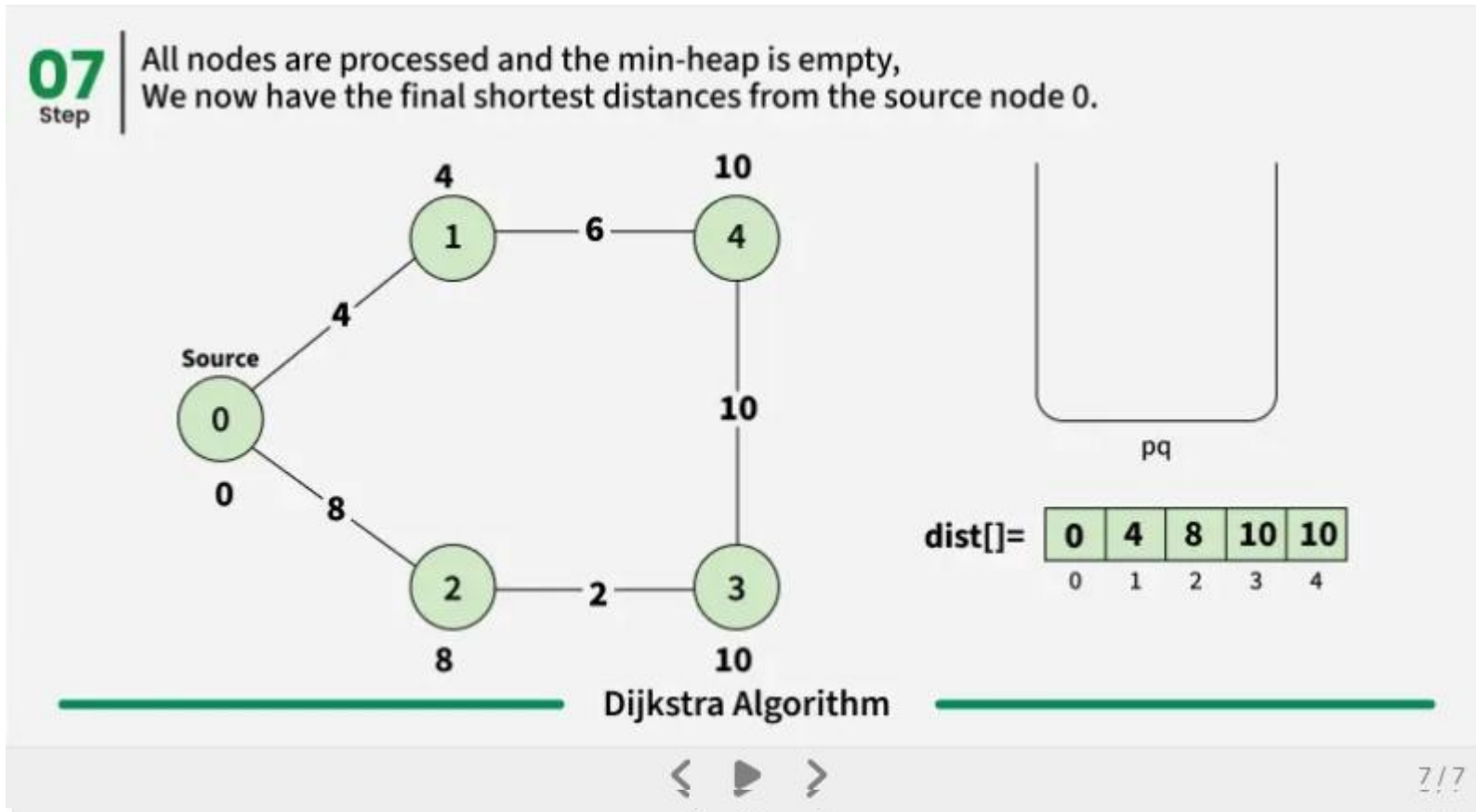
0 to 1 = 4. $0 \rightarrow 1$

0 to 2 = 8. $0 \rightarrow 2$

0 to 3 = 10. $0 \rightarrow 2 \rightarrow 3$

0 to 4 = 10. $0 \rightarrow 1 \rightarrow 4$

Illustration



Space and Time Complexity

- **Time Complexity:** $O(E \cdot \log V)$, Where E is the number of edges and V is the number of vertices.
- **Auxiliary Space:** $O(V)$, Where V is the number of vertices, We do not count the adjacency list in auxiliary space as it is necessary for representing the input graph.



Thank you !!!